DBMS MODULE 3 NOTES

DBMS, CSE (Visvesvaraya Technological University)

Vaishali S R
Assistant Professor
Dept Of CSE,
ACET

COURSE NAME: DATABASE MANAGEMENT SYSTEM

COURSE CODE: BCS403 (2022 Scheme)

SEMESTER: 4

MODULE: 3

NUMBER OF HOURS: 08

CONTENTS:

- **Normalization: Database Design Theory**
  - Informal Design Guidelines For Relation Schema
  - Functional Dependencies
  - Normal Forms Based On Primary Keys
  - Second And Third Normal Forms
  - Boyce-Codd Normal Form
  - Multivalued Dependency
  - Fourth Normal Form
  - Join Dependencies And Fifth Normal Form.

- **SQL**
  - SQL Data Definition And Data Types
  - Specifying Constraints In SQL
  - Retrieval Queries In SQL
  - INSERT, DELETE, And UPDATE Statements In SQL,
  - Additional Features Of SQL

NORMALIZATION DATABASE DESGIN THEORY

Informal Design Guidelines for Relation Schemas

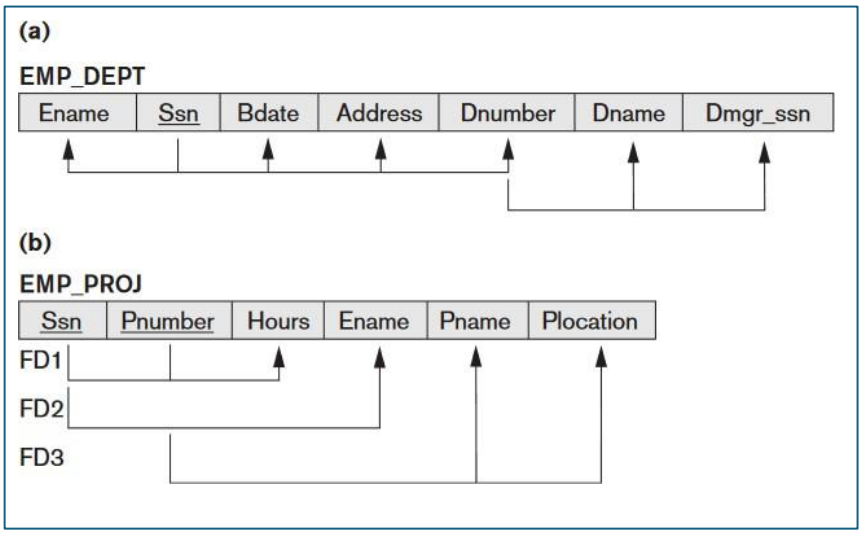
The four informal measures of quality for relation schemas are:

- Semantics of the attributes
- Reducing redundant values in tuples
- Reducing null values in tuples
- Disallowing the possibility of generating spurious tuples

Semantics of Relation Attributes

This section specifies how to interpret the attribute values stored in a tuple of the relation, i.e., how the attribute values in a tuple relate to one another.

Guideline 1: Design a relation schema so that it is easy to explain its meaning. Do not combine attributes from multiple entity types and relationship types into a single relation to save storage space and avoid update, insertion, and deletion anomalies.



Two relation schemas su的的ering from update anomalies.
(a) EMP_DEPT and
(b) EMP_PROJ.

Insertion Anomalies

To insert a new employee tuple into EMP_DEPT, we must include either the attribute values for that department that the employee works for or nulls. It's difficult to insert a new department that has no employee as yet in the EMP_DEPT relation. The only way to do this is to place null values in the attributes for employee. This causes a problem because SSN is the primary key of EMP_DEPT, and each tuple is supposed to represent an employee entity - not a department entity.

Deletion Anomalies

If we delete from EMP_DEPT an employee tuple that happens to represent the last employee working for a particular department, the information concerning that department is lost from the database.

Modification Anomalies

In EMP_DEPT, if we change the value of one of the attributes of a particular department - say the manager of department 5 - we must update the tuples of all employees who work in that department.

Guideline 2:

Design the base relation schemas so that no insertion, deletion, or modification anomalies occur. Reducing the null values in tuples, e.g., if 10% of employees have offices, it is better to have a separate relation, EMP_OFFICE, rather than an attribute OFFICE_NUMBER in EMPLOYEE.

Guideline 3:

- Avoid placing attributes in a base relation whose values are mostly null.
- Disallowing Spurious Tuples

- *Spurious tuples* are tuples that are not in the original relation but are generated by the natural join of decomposed subrelations.

Example:

Decompose EMP_PROJ into EMP_LOCS and EMP_PROJ1.

(a)

EMP_LOCS

Ename	Plocation

P.K.

EMP_PROJ1

Ssn	Pnumber	Hours	Pname	Plocation

P.K.

Figure 14.5
Particularly poor design for the EMP_PROJ relation in Figure 14.3(b). (a) The two relation schemas EMP_LOCS and EMP_PROJ1. (b) The result of projecting the extension of EMP_PROJ from Figure 14.4 onto the relations EMP_LOCS and EMP_PROJ1.

(b)

EMP_LOCS

Ename	Plocation
Smith, John B.	Bellaire
Smith, John B.	Sugarland
Narayan, Ramesh K.	Houston
English, Joyce A.	Bellaire
English, Joyce A.	Sugarland
Wong, Franklin T.	Sugarland
Wong, Franklin T.	Houston
Wong, Franklin T.	Stafford
Zelaya, Alicia J.	Stafford
Jabbar, Ahmad V.	Stafford
Wallace, Jennifer S.	Stafford
Wallace, Jennifer S.	Houston
Borg, James E.	Houston

EMP_PROJ1

Ssn	Pnumber	Hours	Pname	Plocation
123456789	1	32.5	ProductX	Bellaire
123456789	2	7.5	ProductY	Sugarland
666884444	3	40.0	ProductZ	Houston
453453453	1	20.0	ProductX	Bellaire
453453453	2	20.0	ProductY	Sugarland
333445555	2	10.0	ProductY	Sugarland
333445555	3	10.0	ProductZ	Houston
333445555	10	10.0	Computerization	Stafford
333445555	20	10.0	Reorganization	Houston
999887777	30	30.0	Newbenefits	Stafford
999887777	10	10.0	Computerization	Stafford
987987987	10	35.0	Computerization	Stafford
987987987	30	5.0	Newbenefits	Stafford
987654321	30	20.0	Newbenefits	Stafford
987654321	20	15.0	Reorganization	Houston
888665555	20	NULL	Reorganization	Houston

Guideline 4: Design relation schemas so that they can be naturally joined on primary keys or foreign keys in a way that guarantees no spurious tuples are generated.

A functional dependency (FD) is a constraint between two sets of attributes from the database. It is denoted by: $X \rightarrow Y$

We say that "Y is functionally dependent on X". Also, X is called the left-hand side of the FD, and Y is called the right-hand side of the FD.

A functional dependency is a property of the semantics or meaning of the attributes, i.e., a property of the relation schema. They must hold on all relation states (extensions) of R. Relation extensions r(R). A FD $X \rightarrow Y$ is a full functional dependency if removal of any attribute from X means that the dependency does not hold anymore; otherwise, it is a partial functional dependency.

Examples:

1. SSN → ENAME
2. PNUMBER → {PNAME, PLOCATION}
3. {SSN, PNUMBER} → HOURS

FD is a property of the relation schema R, not of a particular relation state/instance.
Let R be a relation schema, where $X \in R$ and $Y \in R$.
For any tuples $t1, t2 \in r$, if $t1[X]=t2[X]$, then $t1[Y]=t2[Y]$.

The FD X→Y holds on R if and only if for all possible relations r(R), whenever two tuples of r agree on the attributes of X, they also agree on the attributes of Y.

- The single arrow denotes "functional dependency".
- X→Y can also be read as "X determines Y".
- The double arrow denotes "logical implication".

Inference Rules

IR1. Reflexivity (e.g., X→X):

- A formal statement of trivial dependencies; useful for derivations.
- If a dependency holds, then we can freely expand its left-hand side.
- The "most powerful" inference rule; useful in multi-step derivations.

Armstrong Inference Rules:

- **Soundness**: Given a set of functional dependencies F specified on a relation schema RR, any dependency that we can infer from F by using IR1 through IR3 holds for every relation state r of R that specifies the dependencies in F. In other words, rules can be used to precisely derive the closure, or no additional FD can be derived.
- **Completeness**: Using IR1 through IR3 repeatedly to infer dependencies until no more dependencies can be inferred results in the complete set of all possible dependencies that can be inferred from F. In other words, given a set of FDs, all implied FDs can be derived using these three rules.

Closure of a Set of Functional Dependencies: Given a set X of FDs in relation RR, the set of all FDs that are implied by X is called the closure of X, denoted as X^+.

Algorithms for Determining X^+:

```
X+:=X;
 Repeat
 oldX+ := X+;
for each FD Y→Z in F do
 if Y⊆X+ then X+:=X+ ∪ Z;
 until old X+ = X+;
```

Example: Given the set of functional dependencies:

A→BC
E→CF
B→E
CD→EF

Compute A,B+ of the set of attributes under this set of FDs.

Solution:

Step 1:

$\{A,B\}^+ := \{A,B\}$.

Go round the inner loop 4 times, once for each of the given FDs.

On the first iteration, for $A \rightarrow BC$: $A \subseteq \{A,B\}$

A,B+:={A,B,C}.

Step 2:

On the second iteration, for $E \rightarrow CF$, $C \notin \{A,B,C\}$.

Step 3:

On the third iteration, for $B \rightarrow E$: $B \subseteq \{A,B,C\}$

$A,B^+ := \{A,B,C,E\}$.

Step 4:

On the fourth iteration, for $CD \rightarrow EF$, EF remains unchanged.

Go round the inner loop 4 times again.

On the first iteration, the result does not change;on the second, it expands to {A,B,C,E,F}; On the third and fourth, it does not change. Now go round the inner loop 4 times. The closure does not change, and so the whole process terminates, with

A,B+={A,B,C,E,F}.

Example:

Given
F={SSN ENAME,PNUMBER→{PNAME, PLOCATION},{SSN, PNUMBER}→HOURS }

$\{SSN\}^+ = \{SSN, ENAME\}$

$\{PNUMBER\}^+ = ?$

$\{SSN,PNUMBER\}^+ = ?$

Normalization

Purpose of Normalization:

- Addressing problems associated with redundant data.
- Identifying various types of update anomalies such as insertion, deletion, and modification anomalies.
- Recognizing the appropriateness or quality of the design of relations.
- Understanding the concept of functional dependency, the main tool for measuring the appropriateness of attribute groupings in relations.
- Utilizing functional dependencies to group attributes into relations that are in a known normal form.
- Defining normal forms for relations.
- Undertaking the process of normalization.
- Identifying commonly used normal forms: first (1NF), second (2NF), and third (3NF) normal forms, and Boyce-Codd normal form (BCNF).
- Identifying fourth (4NF) and fifth (5NF) normal forms.

Main Objective: The main objective in developing a logical data model for relational database systems is to create an accurate representation of the data, its relationships, and constraints. To achieve this objective, we must identify a suitable set of relations. A technique for producing a set of relations with desirable properties, given the data requirements of an enterprise.

NORMAL FORMS: A relation is defined as a set of tuples. By definition, all elements of a set are distinct; hence, all tuples in a relation must also be distinct. This means that no two tuples can have the same combination of values for all their attributes.

Superkeys and Keys:

- Any set of attributes of a relation schema is called a superkey. Every relation has at least one superkey—the set of all its attributes.
- A key is a minimal superkey, i.e., a superkey from which we cannot remove any attribute and still have the uniqueness constraint hold.
- A relation schema may have more than one key. In this case, each of the keys is called a candidate key. One of the candidate keys is typically designated as the primary key of the relation.
- A foreign key is a key in a relation R but not a key (just an attribute) in another relation R′ of the same schema.

Integrity Constraints:

- The entity integrity constraint states that no primary key value can be null. This ensures that the primary key value is used to identify individual tuples in a relation.
- The referential integrity constraint is specified between two relations and maintains consistency among tuples of the two relations. It ensures that a tuple in one relation that refers to another relation must refer to an existing tuple in that relation.

Prime and Nonprime Attributes:

- An attribute of a relation schema R is called a prime attribute of the relation R if it is a member of any key of the relation R.
- An attribute is called nonprime if it is not a prime attribute, meaning it is not a member of any candidate key.

Normalization:

- The goal of normalization is to create a set of relational tables that are free of redundant data and can be consistently and correctly modified.
- All tables in a relational database should ideally be in the third normal form (3NF).
- Normalization is a process during which unsatisfactory relation schemas are decomposed by breaking up their attributes into smaller relation schemas that possess desirable properties.
- One objective of the original normalization process is to ensure that update anomalies such as insertion, deletion, and modification anomalies do not occur.

The Most Commonly Used Normal Forms:

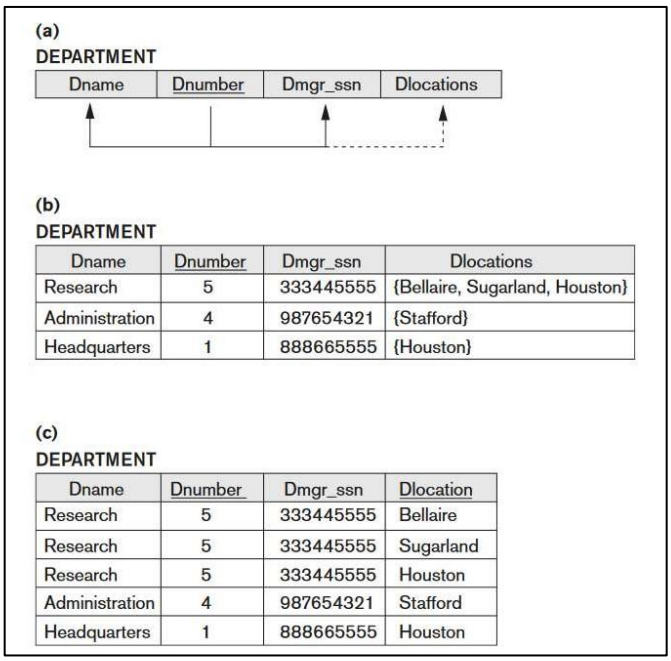
- **First Normal Form (1NF)**
- **Second Normal Form (2NF)**
- **Third Normal Form (3NF)**
- **Boyce-Codd Normal Form (BCNF)**

Other Normal Forms:

- **Fourth Normal Form (4NF)**
- **Fifth Normal Form (5NF)**
- **Domain Key Normal Form (DK/NF)**

First Normal Form (1NF)

First normal form is now considered to be part of the formal definition of a relation; historically, it was defined to disallow multivalued attributes, composite attributes, and their combinations. It states that the domains of attributes must include only atomic (simple, indivisible) values and that the value of any attribute in a tuple must be a single value from the domain of that attribute.



(a)
DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations

(b)
DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)
DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocation
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

(a) A relation schema that is not in 1NF. (b)Sample state of relation DEPARTMENT.(c) 1NF version of the same relation with redundancy

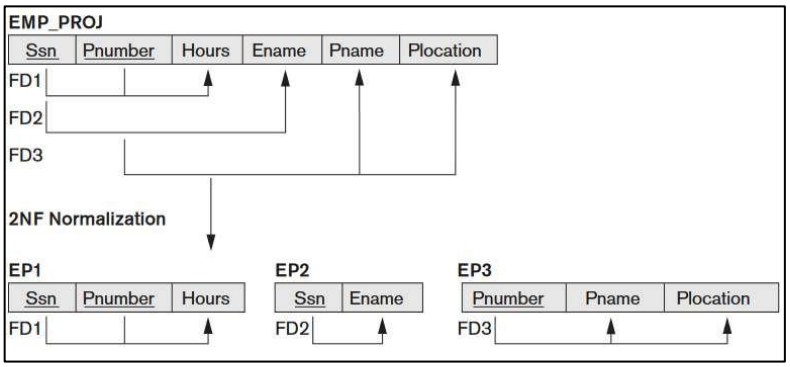
Practical Rule: "Eliminate Repeating Groups," i.e., make a separate table for each set of related attributes, and give each table a primary key.

Formal Definition: A relation is in first normal form (1NF) if and only if all underlying simple domains contain atomic values only.

Second Normal Form (2NF)

Second normal form is based on the concept of fully functional dependency. A functional dependency X→Y is fully functional if removal of any attribute A from X means that the dependency does not hold anymore.

A relation schema is in 2NF if every nonprime attribute in the relation is fully functionally dependent on the primary key of the relation. It can also be restated as: a relation schema is in 2NF if every nonprime attribute in the relation is not partially dependent on any key of the relation.

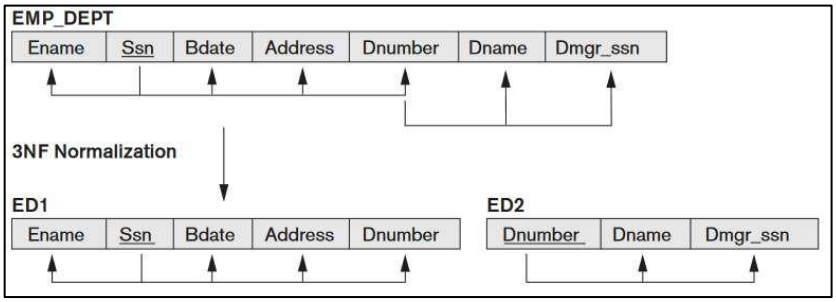


Practical Rule: "Eliminate Redundant Data," i.e., if an attribute depends on only part of a multivalued key, remove it to a separate table.

Formal Definition: A relation is in second normal form (2NF) if and only if it is in 1NF and every nonkey attribute is fully dependent on the primary key.

Third Normal Form (3NF)

Third normal form is based on the concept of transitive dependency. A functional dependency X→Y in a relation is a transitive dependency if there is a set of attributes Z that is not a subset of any key of the relation, and both X→Z and Z→Y hold. In other words, a relation is in 3NF if, whenever a functional dependency X→A holds in the relation, either (a) X is a superkey of the relation, or (b) A is a prime attribute of the relation.



Practical Rule: "Eliminate Columns not Dependent on Key," i.e., if attributes do not contribute to a description of a key, remove them to a separate table.

Formal Definition: A relation is in third normal form (3NF) if and only if it is in 2NF and every nonkey attribute is nontransitively dependent on the primary key.

1NF: R is in 1NF iff all domain values are atomic.

2NF: R is in 2NF iff R is in 1NF and every nonkey attribute is fully dependent on the key.

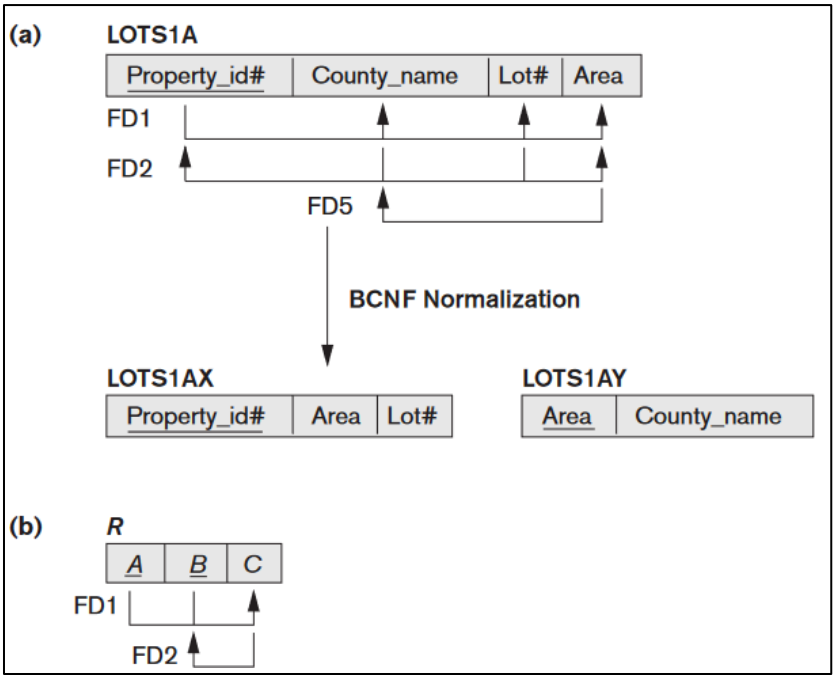
3NF: R is in 3NF if R is in 2NF and every nonkey attribute is non-transitively dependent on the key.

Boyce-Codd Normal Form (BCNF):

A relation schema R is in Boyce-Codd Normal Form (BCNF) if whenever an FD X -> A holds in R, then X is a superkey of R. Each normal form is strictly stronger than the previous one:

- Every 2NF relation is in 1NF
- Every 3NF relation is in 2NF
- Every BCNF relation is in 3NF
- There exist relations that are in 3NF but not in BCNF.
- A relation is in BCNF if and only if every determinant is a candidate key.

Additional criteria may be needed to ensure the set of relations in a relational database is satisfactory.



(a) BCNF normalization of LOTS1A with the functional dependency FD2 being lost in the decomposition.

(b) A schematic relation with FDs; it is in 3NF, but not in BCNF due to the FD C → B.

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

A relation TEACH that is in 3NF but not BCNF.

If XY is non-trivial, then X is a superkey.

STREET	CITY	ZIP

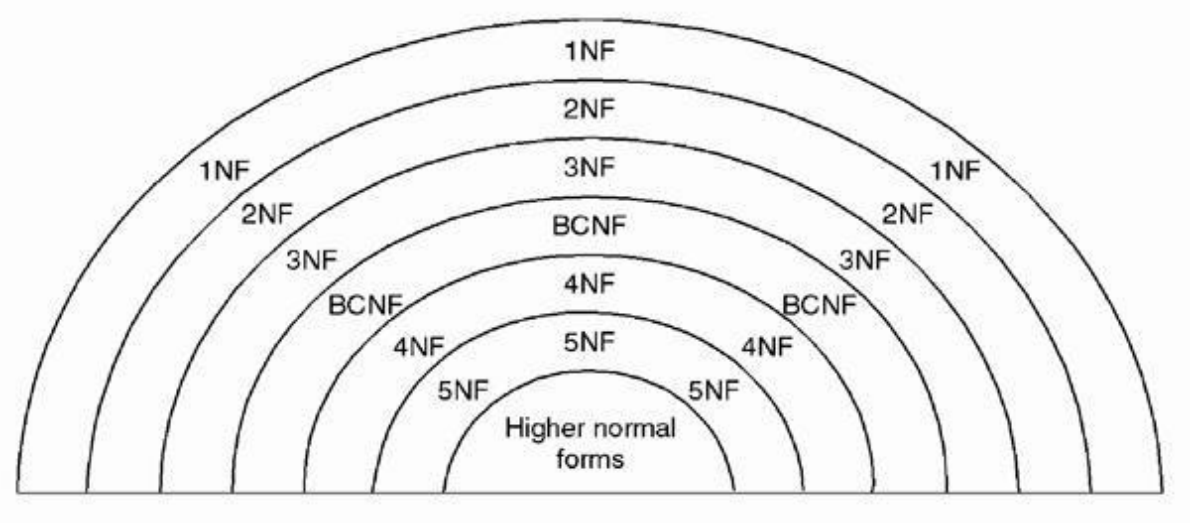
- {CITY, STREET} →ZIP
- ZIP→CITY

Insertion anomaly: The city of a zip code can't be stored if the street is not given.

Normalization:

STREET	ZIP	ZIP	CITY

Relationship Between Normal Forms



Multivalued Dependencies and Fourth Normal Form (4NF)

4NF is associated with a dependency called multi-valued dependency (MVD). MVDs in a relation stem from first normal form (1NF), which prohibits an attribute in a row from having a set of values. An MVD signifies a dependency between attributes (e.g., A, B, and C) in a relation, such that for each value of A, there is a set of values for B and a set of values for C. However, the sets of values for B and C are independent of each other.

MVD between attributes A, B, and C in a relation is represented using the following notation:

- A ->> B (A multidetermines B)
- A ->> C

Formal Definition of Multivalued Dependency: A multivalued dependency (MVD) X ->> Y specified on R, where X and Y are both subsets of R and Z = (R - (X U Y)), specifies the following restrictions on r(R):

- t3[X] = t4[X] = t1[X] = t2[X]
- t3[Y] = t1[Y] and t4[Y] = t2[Y]
- t3[Z] = t2[Z] and t4[Z] = t1[Z]

Fourth Normal Form (4NF): A relation in Boyce-Codd Normal Form (BCNF) and containing no MVDs. Transforming from BCNF to 4NF involves removing the MVD from the relation by placing the attribute(s) in a new relation along with a copy of the determinant(s).

A Relation is in 4NF if it is in 3NF and there is no multivalued dependencies.

Join Dependencies and 5 NF

A join dependency (JD), denoted by JD{R1, R2, …, Rn}, specified on relation schema R, specifies a constraint on the states r of R. The constraint states that every legal state r of R should have a lossless join decomposition into R1, R2, …, Rn; that is, for every such r we have:

$(\pi R1(r), (\pi R2(r), \ldots, (\pi Rn(r))) = r$

Lossless-join property refers to when we decompose a relation into two relations - we can rejoin the resulting relations to produce the original relation. However, sometimes there is the requirement to decompose a relation into more than two relations. Although rare, these cases are managed by join dependency and 5NF.

5NF (or project-join normal form (PJNF))

A relation that has no join dependency.

The Project-Join Normal Form (PJNF), also known as the Fifth Normal Form (5NF), is a less common normal form compared to the first through fourth normal forms. PJNF is particularly useful in complex database design scenarios, where multiple many-to-many relationships exist between entities.

In PJNF, each table contains only attributes that are functionally dependent on the primary key. This means that every non-key attribute is dependent on all the components of the primary key, rather than just a part of it. Additionally, PJNF allows for a more flexible representation of multi-valued dependencies than earlier normal forms.

To achieve PJNF, you may need to decompose your tables further, often resulting in more tables in your schema. However, this decomposition ensures that your database design accurately reflects the complex relationships between entities, leading to better data integrity and flexibility.

PJNF or 5NF is a normal form used in database normalization to ensure that each table accurately represents the relationships between entities and attributes, particularly in scenarios involving complex many-to-many relationships.

(a) EMP

Ename	Pname	Dname
Smith	X	John
Smith	Y	Anna
Smith	X	Anna
Smith	Y	John

(c) SUPPLY

Sname	Part_name	Proj_name
Smith	Bolt	ProjX
Smith	Nut	ProjY
Adamsky	Bolt	ProjY
Walton	Nut	ProjZ
Adamsky	Nail	ProjX
Adamsky	Bolt	ProjX
Smith	Bolt	ProjY

(b) EMP_PROJECTS

Ename	Pname
Smith	X
Smith	Y

EMP_DEPENDENTS

Ename	Dname
Smith	John
Smith	Anna

(d) R_1

Sname	Part_name
Smith	Bolt
Smith	Nut
Adamsky	Bolt
Walton	Nut
Adamsky	Nail

R_2

Sname	Proj_name
Smith	ProjX
Smith	ProjY
Adamsky	ProjY
Walton	ProjZ
Adamsky	ProjX

R_3

Part_name	Proj_name
Bolt	ProjX
Nut	ProjY
Bolt	ProjY
Nut	ProjZ
Nail	ProjX

Figure 14.15
Fourth and fifth normal forms.
(a) The EMP relation with two MVDs: Ename $\twoheadrightarrow$ Pname and Ename $\twoheadrightarrow$ Dname.
(b) Decomposing the EMP relation into two 4NF relations EMP_PROJECTS and EMP_DEPENDENTS.
(c) The relation SUPPLY with no MVDs is in 4NF but not in 5NF if it has the JD(R_1, R_2, R_3).
(d) Decomposing the relation SUPPLY into the 5NF relations R_1, R_2, R_3.

Other Dependencies

Template Dependencies

The concept behind template dependencies is to define a template—or example—that specifies each constraint or dependency. There are two types of templates: tuple-generating templates and constraint-generating templates. A template comprises several hypothesis tuples meant to demonstrate examples of tuples that may appear in one or more relations. The other component of the template is the template conclusion. For tuple-generating templates, the conclusion is a set of tuples that must also exist in the relations if the hypothesis tuples are present. For constraint-generating templates, the template conclusion is a condition that must hold on the hypothesis tuples.

Domain Key Normal Form

The concept behind domain-key normal form (DKNF) is to theoretically specify the "ultimate normal form" that considers all possible types of dependencies and constraints.

A relation is considered to be in **DKNF** if all constraints and dependencies that should hold on the relation can be enforced simply by enforcing the domain constraints and key constraints on the relation.

However, due to the challenge of incorporating complex constraints into a DKNF relation, its practical utility is limited. This limitation arises because it may be quite difficult to specify general integrity constraints.

For instance, consider a relation (where VIN# is the vehicle identification number) and another relation MANUFACTURE(VIN#, COUNTRY) (where COUNTRY is the country of manufacture).

A general constraint may be of the following form: "If the MAKE is either Toyota or Lexus, then the first character of the VIN# is a 'J' if the country of manufacture is Japan; if the MAKE is Honda or Acura, the second character of the VIN# is a 'J' if the country of manufacture is Japan." There is no simplified way to represent such constraints short of writing a procedure (or general assertions) to test them.

MODULE 3

SQL

Introduction

SQL was called SEQUEL (Structured English Query Language) and was designed and implemented at IBM Research. The SQL language may be considered one of the major reasons forthe commercial success of relational databases. SQL is a comprehensive database language. It hasstatements for data definitions, queries, and updates. Hence, it is both a DDL *and* a DML. In addition, it has facilities for defining views on the database, for specifying security and authorization, for defining integrity constraints, and for specifying transaction controls. It also has rules for embedding SQL statements into a general-purpose programming language such as Java, COBOL, or C/C++.

SQL Data Definition and Data Types

SQL uses the terms table, row, and column for the formal relational model terms relation, tuple, and attribute, respectively. The main SQL command for data definition is the CREATE statement, which can be used to create schemas, tables (relations), domains, views, assertions and triggers.

Schema and Catalog Concepts in SQL

An SQL schema is identified by a schema name, and includes an authorization identifier to indicate the user or account who owns the schema, as well as descriptors for *each element* in the schema. Schema elements include tables, constraints, views, domains, and other constructs (such as authorization grants) that describe the schema. A schema is created via the CREATE SCHEMA statement .

For example, the following statement creates a schema called COMPANY, owned by the user with authorization identifier 'Jsmith'..

CREATE SCHEMA COMPANY **AUTHORIZATION** 'Jsmith';

In general, not all users are authorized to create schemas and schema elements. The privilege to create schemas, tables, and other constructs must be explicitly granted to the relevant user accounts by the system administrator or DBA.

SQL uses the concept of a **catalog** —a named collection of schemas in an SQL environment. A catalog always contains a special schema called INFORMATION_SCHEMA, which provides information on all the schemas in the catalog and all the element descriptors in these

schemas. Integrity constraints such as referential integrity can be defined between relations only if they exist in schemas within the same catalog. Schemas within the same catalog can also share certain elements, such as domain definitions.

The CREATE TABLE Command in SQL

The CREATE TABLE command is used to specify a new relation by giving it a name and specifying its attributes and initial constraints. The attributes are specified first, and each attribute is given a name, a data type to specify its domain of values, and any attribute constraints, such as NOT NULL. The key, entity integrity, and referential integrity constraints can be specified within the CREATE TABLE statement after the attributes are declared, or they can be added later using the ALTER TABLE command.

Typically, the SQL schema in which the relations are declared is implicitly specified in the environment in which the CREATE TABLE statements are executed. Alternatively, we can explicitly attach the schema name to the relation name, separated by a period. For example, by writing

CREATE TABLE COMPANY.EMPLOYEE ...

rather than

CREATE TABLE EMPLOYEE ...

The relations declared through CREATE TABLE statements are called **base tables.**

Examples:

```
CREATE TABLE EMPLOYEE
        ( Fname          VARCHAR(15)          NOT NULL,
          Minit          CHAR,
          Lname          VARCHAR(15)          NOT NULL,
          Ssn            CHAR(9)              NOT NULL,
          Bdate          DATE,
          Address        VARCHAR(30),
          Sex            CHAR,
          Salary         DECIMAL(10,2),
          Super_ssn      CHAR(9),
          Dno            INT                  NOT NULL,
        PRIMARY KEY (Ssn),
        FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn),
        FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber) );
```

```
CREATE TABLE DEPARTMENT
        ( Dname                 VARCHAR(15)             NOT NULL,
          Dnumber               INT                     NOT NULL,
          Mgr_ssn               CHAR(9)                 NOT NULL,
          Mgr_start_date        DATE,
        PRIMARY KEY (Dnumber),
        UNIQUE (Dname),
        FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
CREATE TABLE DEPT_LOCATIONS
        ( Dnumber               INT                     NOT NULL,
          Dlocation             VARCHAR(15)             NOT NULL,
        PRIMARY KEY (Dnumber, Dlocation),
        FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
CREATE TABLE PROJECT
        ( Pname                 VARCHAR(15)             NOT NULL,
          Pnumber               INT                     NOT NULL,
          Plocation             VARCHAR(15),
          Dnum                  INT                     NOT NULL,
        PRIMARY KEY (Pnumber),
        UNIQUE (Pname),
        FOREIGN KEY (Dnum) REFERENCES DEPARTMENT(Dnumber) );

CREATE TABLE WORKS_ON
        ( Essn                  CHAR(9)                 NOT NULL,
          Pno                   INT                     NOT NULL,
          Hours                 DECIMAL(3,1)            NOT NULL,
        PRIMARY KEY (Essn, Pno),
        FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn),
        FOREIGN KEY (Pno) REFERENCES PROJECT(Pnumber) );
CREATE TABLE DEPENDENT
        ( Essn                  CHAR(9)                 NOT NULL,
          Dependent_name        VARCHAR(15)             NOT NULL,
          Sex                   CHAR,
          Bdate                 DATE,
          Relationship          VARCHAR(8),
        PRIMARY KEY (Essn, Dependent_name),
        FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn) );
```

Attribute Data Types and Domains in SQL

Basic data types

1. **Numeric** data types includes
   - integer numbers of various sizes (INTEGER or INT, and SMALLINT)
   - floating-point (real) numbers of various precision (FLOAT or REAL, and DOUBLE PRECISION).
   - Formatted numbers can be declared by using DECIMAL(i,j) —or DEC(i,j) or NUMERIC(i,j) —where
     i - precision, total number of decimal digits
     j - scale, number of digits after the decimal point

2. **Character-string data types**
   - fixed length —CHAR(*n*) or CHARACTER(*n*), where *n* is the number of characters
   - varying length —VARCHAR(*n*) or CHAR VARYING(*n*) or CHARACTER VARYING(*n*), where *n* is the maximum number of characters
   - When specifying a literal string value, it is placed between single quotation marks (apostrophes), and it is *case sensitive*
   - For fixed length strings, a shorter string is padded with blank characters to the right
   - For example, if the value 'Smith' is for an attribute of type CHAR(10), it is padded with five blank characters to become 'Smith ' if needed
   - Padded blanks are generally ignored when strings are compared
   - Another variable-length string data type called CHARACTER LARGE OBJECT or CLOB is also available to specify columns that have large text values, such as documents
   - The CLOB maximum length can be specified in kilobytes (K), megabytes (M), or gigabytes (G)
   - For example, CLOB(20M) specifies a maximum length of 20 megabytes.

3. **Bit-string** data types are either of
   - fixed length *n* —BIT(*n*)—or varying length —BIT VARYING(*n*), where *n* is the maximum number of bits.
   - The default for *n*, the length of a character string or bit string, is 1.

- Literal bit strings are placed between single quotes but preceded by a B to distinguish them from character strings; for example, B‘10101’
- Another variable-length bitstring data type called BINARY LARGE OBJECT or BLOB is also available to specify columns that have large binary values, such as images.
- The maximum length of a BLOB can be specified in kilobits (K), megabits (M), or gigabits (G)
- For example, BLOB(30G) specifies a maximum length of 30 gigabits.

4. **A Boolean** data type has the traditional values of TRUE or FALSE.In SQL, because of the presence of NULL values, a three-valued logic is used, so a third possible value for a Boolean data type is UNKNOWN
5. The **DATE** data type has ten positions, and its components are YEAR, MONTH, and DAY in the form YYYY-MM-DD
6. The **TIME data** type has at least eight positions, with the components HOUR, MINUTE, and SECOND in the form HH:MM:SS.
   Only valid dates and times should be allowed by the SQL implementation.
7. **TIME WITH TIME ZONE** data type includes an additional six positions for specifying the displacement from the standard universal time zone, which is in the range +13:00 to–12:59 in units of HOURS:MINUTES. If WITH TIME ZONE is not included, the default is the local time zone for the SQL session.

Additional data types

1. **Timestamp** data type (TIMESTAMP) includes the DATE and TIME fields, plus a minimum of six positions for decimal fractions of seconds and an optional WITH TIME ZONE qualifier.
2. **INTERVA**L data type. This specifies an **interva l a** relative value that can be used to increment or decrement an absolute value of a date, time, or timestamp. Intervals are qualified to be either YEAR/MONTH intervals or DAY/TIME intervals.

It is possible to specify the data type of each attribute directly or a domain can be declared, and the domain name used with the attribute Specification. This makes it easier to change the data type for a domain that is used by numerous attributes in a schema, and improves schema readability. For example, we can create a domain SSN_TYPE by the following statement:

CREATE DOMAIN SSN_TYPE **AS** CHAR(9);

We can use SSN_TYPE in place of CHAR(9) for the attributes Ssn and Super_ssn of EMPLOYEE, Mgr_ssn of DEPARTMENT, Essn of WORKS_ON, and Essn of DEPENDENT

Specifying Constraints in SQL

Basic constraints that can be specified in SQL as part of table creation:

- key and referential integrity constraints
- Restrictions on attribute domains and NULLs
- constraints on individual tuples within a relation

Specifying Attribute Constraints and Attribute Defaults

Because SQL allows NULLs as attribute values, a constraint NOT NULL may be specified if NULL is not permitted for a particular attribute. This is always implicitly specified for the attributes that are part of the primary key of each relation, but it can be specified for any other attributes whose values are required not to be NULL.

It is also possible to define a default value for an attribute by appending the clause **DEFAULT** <value> to an attribute definition. The default value is included in any new tuple if an explicit value is not provided for that attribute.

```
CREATE TABLE DEPARTMENT
( . . . ,
    Mgr_ssn CHAR(9) NOT NULL DEFAULT '888665555',
    _ _ _ _ _ _ _
    _ _ _ _ _ _ _ _
)
```

Another type of constraint can restrict attribute or domain values using the **CHECK** clause following an attribute or domain definition . For example, suppose that department numbers are restricted to integer numbers between 1 and 20; then, we can change the attribute declaration of Dnumber in the DEPARTMENT table to the following:

Dnumber INT **NOT NULL CHECK** (Dnumber > 0 **AND** Dnumber < 21);

The CHECK clause can also be used in conjunction with the CREATE DOMAIN statement.For example, we can write the following statement:

CREATE DOMAIN D_NUM **AS** INTEGER

CHECK (D_NUM > 0 **AND** D_NUM < 21);

We can then use the created domain D_NUM as the attribute type for all attributes that refer to department number such as Dnumber of DEPARTMENT, Dnum of PROJECT, Dno of EMPLOYEE, and so on.

Specifying Key and Referential Integrity Constraints

The **PRIMARY KEY** clause specifies one or more attributes that make up the primary key of a relation. If a primary key has a single attribute, the clause can follow the attribute directly. For example, the primary key of DEPARTMENT can be specified as:

Dnumber INT **PRIMARY KEY**;

The **UNIQUE** clause can also be specified directly for a secondary key if the secondary key is a single attribute, as in the following example:

Dname VARCHAR(15) **UNIQUE**;

Referential integrity is specified via the **FOREIGN KEY** clause

FOREIGN KEY (Super_ssn) **REFERENCES** EMPLOYEE(Ssn),
FOREIGN KEY (Dno) **REFERENCES** DEPARTMENT(Dnumber

A referential integrity constraint can be violated when tuples are inserted or deleted, or when a foreign key or primary key attribute value is modified. The default action that SQL takes for an integrity violation is to **reject** the update operation that will cause a violation, which is known as the RESTRICT option.

The schema designer can specify an alternative action to be taken by attaching a **referential triggered action** clause to any foreign key constraint. The options include SET NULL, CASCADE, and SET DEFAULT. An option must be qualified with either ON DELETE or ON UPDATE

- **FOREIGN KEY**(Dno) **REFERENCES** DEPARTMENT(Dnumber) **ON DELETE** SET DEFAULT **ON UPDATE** CASCADE
- **FOREIGN KEY** (Super_ssn) **REFERENCES** EMPLOYEE(Ssn) **ON DELETE** SET NULL **ON UPDATE** CASCADE
- **FOREIGN KEY** (Dnumber) **REFERENCES** DEPARTMENT(Dnumber) **ON DELETE** CASCADE **ON UPDATE** CASCADE

In general, the action taken by the DBMS for SET NULL or SET DEFAULT is the same for both ON DELETE and ON UPDATE: The value of the affected referencing attributes is changed to NULL for SET NULL and to the specified default value of the referencing attribute for SET DEFAULT.

The action for CASCADE ON DELETE is to delete all the referencing tuples whereas the action for CASCADE ON UPDATE is to change the value of the referencing foreign key attribute(s) to the updated (new) primary key value for all the referencing tuples . It is the responsibility of the database designer to choose the appropriate action and to specify it in the database schema. As a general rule, the CASCADE option is suitable for "relationship" relations such as WORKS_ON; for relations that represent multivalued attributes, such as DEPT_LOCATIONS; and for relations that represent weak entity types, such as DEPENDENT.

Giving Names to Constraints

The names of all constraints within a particular schema must be unique. A constraint name is used to identify a particular constraint in case the constraint must be dropped later and replaced with another constraint.

Specifying Constraints on Tuples Using CHECK

In addition to key and referential integrity constraints, which are specified by special keywords, other *table constraints* can be specified through additional CHECK clauses at the end of a CREATE TABLE statement. These can be called **tuple-based** constraints because they apply to each tuple individually and are checked whenever a tuple is inserted or modified

For example, suppose that the DEPARTMENT table had an additional attribute Dept_create_date, which stores the date when the department was created. Then we could add the following CHECK clause at the end of the CREATE TABLE statement for the DEPARTMENT table to make sure that a manager's start date is later than the department creation date

CHECK (Dept_create_date <= Mgr_start_date);

Basic Retrieval Queries in SQL

SQL has one basic statement for retrieving information from a database: the **SELECT** statement.

The SELECT-FROM-WHERE Structure of Basic SQL Queries

The basic form of the SELECT statement, sometimes called a **mapping** or a **select-from-where block**, is formed of the three clauses SELECT, FROM, and WHERE and has the following form:

SELECT <attribute list>

FROM <table list>

WHERE <condition>;

Where,

- <attribute list> is a list of attribute names whose values are to be retrieved by the query
- <table list> is a list of the relation names required to process the query
- <condition> is a conditional (Boolean) expression that identifies the tuples to be retrieved by the query.

Examples:

1. Retrieve the birth date and address of the employee(s) whose name is 'John B. Smith'.

```
SELECT Bdate, Address
FROM EMPLOYEE
WHERE Fname='John' AND Minit='B' AND Lname='Smith';
```

The SELECT clause of SQL specifies the attributes whose values are to be retrieved, which are called the **projection attributes.** The WHERE clause specifies the Boolean condition that must be true for any retrieved tuple, which is known as the **selection condition.**

2. Retrieve the name and address of all employees who work for the 'Research' department.

```
SELECT Fname, Lname, Address
FROM EMPLOYEE, DEPARTMENT
WHERE Dname='Research' AND Dnumber=Dno;
```

In the WHERE clause, the condition Dname = 'Research' is a **selection condition** that chooses the particular tuple of interest in the DEPARTMENT table, because Dname is an attribute of DEPARTMENT. The condition Dnumber = Dno is called a **join condition**, because it combines two tuples: one from DEPARTMENT and one from EMPLOYEE, whenever the value of Dnumber in DEPARTMENT is equal to thevalue of Dno in EMPLOYEE.A query that involves only selection and join conditions plus projection attributes is known as a **select-project-join** query.

3. For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

SELECT Pnumber, Dnum, Lname, Address, Bdate

FROM PROJECT, DEPARTMENT, EMPLOYEE

WHERE Dnum=Dnumber **AND** Mgr_ssn=Ssn **AND** Plocation='Stafford';

The join condition Dnum = Dnumber relates a project tuple to its controlling department tuple, whereas the join condition Mgr_ssn = Ssn relates the controlling department tuple to the employee tuple who manages that department. Each tuple in the result will be a *combination* of one project, one department, and one employee that satisfies the join conditions. The projection attributes are used to choose the attributes to be displayed from each combined tuple.

Ambiguous Attribute Names, Aliasing, Renaming, and Tuple Variables

In SQL, the same name can be used for two or more attributes as long as the attributes are in different relations. If this is the case, and a multitable query refers to two or more attributes with the same name, we must **qualify** the attribute name with the relation name to prevent ambiguity. This is done by prefixing the relation name to the attribute name and separating the two by a period.

Example: Retrieve the name and address of all employees who work for the 'Research' department

SELECT Fname, EMPLOYEE.Name, Address

FROM EMPLOYEE, DEPARTMENT

WHERE DEPARTMENT.Name='Research' **AND**

DEPARTMENT.Dnumber=EMPLOYEE.Dnumber;

The ambiguity of attribute names also arises in the case of queries that refer to the same relation twice. For example consider the query: For each employee, retrieve the employee's first and last name and the first and last name of his or her immediate supervisor.

SELECT E.Fname, E.Lname, S.Fname, S.Lname

FROM EMPLOYEE **AS** E, EMPLOYEE **AS** S

WHERE E.Super_ssn=S.Ssn;

In this case, we are required to declare alternative relation names E and S, called **aliases** or **tuple variables**, for the EMPLOYEE relation. An alias can follow the keyword **AS**, or it can directly follow the relation name—for example, by writing EMPLOYEE E, EMPLOYEE S. It is also possible to **rename** the relation attributes within the query in SQL by giving them aliases. For example, if we write

EMPLOYEE **AS** E(Fn, Mi, Ln, Ssn, Bd, Addr, Sex, Sal, Sssn, Dno)

in the FROM clause, Fn becomes an alias for Fname, Mi for Minit, Ln for Lname, and so on

Unspecified WHERE Clause and Use of the Asterisk

A missing WHERE clause indicates no condition on tuple selection; hence, all tuples of the relation specified in the FROM clause qualify and are selected for the query result.If more than one relation is specified in the FROM clause and there is no WHERE clause, then the CROSS PRODUCT—all possible tuple combinations of these relations is selected.

Example: Select all EMPLOYEE Ssns and all combinations of EMPLOYEE Ssn and DEPARTMENT Dname in the database.

```
SELECT Ssn
FROM EMPLOYEE;
SELECT Ssn, Dname
FROM EMPLOYEE, DEPARTMENT;
```

To retrieve all the attribute values of the selected tuples, we do not have to list the attribute names explicitly in SQL; we just specify an asterisk (*), which stands for all the attributes. For example, the following query retrieves all the attribute values of any EMPLOYEE who works in DEPARTMENT number 5

```
SELECT * FROM EMPLOYEE WHERE Dno=5;

SELECT * FROM EMPLOYEE, DEPARTMENT WHERE Dname='Research'
AND Dno=Dnumber;

SELECT * FROM EMPLOYEE, DEPARTMENT;
```

Tables as Sets in SQL

SQL usually treats a table not as a set but rather as a multiset; duplicate tuples can appear more than once in a table, and in the result of a query. SQL does not automatically eliminate duplicate tuples in the results of queries, for the following reasons:

- Duplicate elimination is an expensive operation. One way to implement it is to sort the tuples first and then eliminate duplicates.
- The user may want to see duplicate tuples in the result of a query.
- When an aggregate function is applied to tuples, in most cases we do not want to eliminate duplicates.

If we do want to eliminate duplicate tuples from the result of an SQL query, we use the keyword **DISTINCT** in the SELECT clause, meaning that only distinct tuples should remain in the result.

Example : Retrieve the salary of every employee and all distinct salary values

(a) SELECT ALL Salary **FROM** EMPLOYEE;

(b) SELECT DISTINCT Salary **FROM** EMPLOYEE;

(a)

Salary
30000
40000
25000
43000
38000
25000
25000
55000

(b)

Salary
30000
40000
25000
43000
38000
55000

SQL has directly incorporated some of the set operations from mathematical *set theory*, which are also part of relational algebra. There are

- set union (**UNION**)
- set difference (**EXCEPT)** and
- set intersection (**INTERSECT**)

The relations resulting from these set operations are sets of tuples; that is, duplicate tuples are eliminated from the result. These set operations apply only to union-compatible relations, so we must make sure that the two relations on which we apply the operation have the same attributes and that the attributes appear in the same order in both relations.

Example: Make a list of all project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project

```
(SELECT DISTINCT Pnumber FROM PROJECT, DEPARTMENT,
 EMPLOYEE WHERE Dnum=Dnumber AND Mgr_ssn=Ssn AND Lname='Smith' )
 UNION
( SELECT DISTINCT Pnumber FROM PROJECT, WORKS_ON, EMPLOYEE
WHERE Pnumber=Pno AND Essn=Ssn AND Lname='Smith' );
```

Substring Pattern Matching and Arithmetic Operators

Several more features of SQL

The first feature allows comparison conditions on only parts of a character string, using the **LIKE** comparison operator. This can be used for string **pattern matching**. Partial strings are specified using two reserved characters:

- % replaces an arbitrary number of zero or more characters
- _ (underscore) replaces a single character

For example, consider the following query: Retrieve all employees whose address is in Houston, Texas

SELECT Fname, Lname **FROM** EMPLOYEE **WHERE** Address
LIKE '%Houston,TX%';

To retrieve all employees who were born during the 1950s, we can use Query

SELECT Fname, Lname **FROM** EMPLOYEE
WHERE Bdate **LIKE** '___5 _____';

If an underscore or % is needed as a literal character in the string, the character should be preceded by an *escape character*, which is specified after the string using the keyword ESCAPE. For example, 'AB_CD%EF' ESCAPE '\' represents the literal string 'AB_CD%EF' because \ is specified as the escape character. Also, we need a rule to specify apostrophes or single quotation marks (' ') if they are to be included in a string because they are used to begin and end strings. If an apostrophe (') is needed, it is represented as two consecutive apostrophes ('') so that it will not be interpreted as ending the string.

Another feature allows the use of arithmetic in queries. The standard arithmetic operators for addition (+), subtraction (-), multiplication (*), and division (/) can be applied to numeric values or attributes with numeric domains. For example, suppose that we want to see the effect of giving all employees
who work on the 'ProductX' project a 10 percent raise; we can issue the following query:

SELECT E.Fname, E.Lname, 1.1 * E.Salary **AS** Increased_sal
FROM EMPLOYEE **AS** E, WORKS_ON **AS** W, PROJECT **AS** P
WHERE E.Ssn=W.Essn **AND** W.Pno=P.Pnumber **AND** P.Pname='ProductX';

Example: Retrieve all employees in department 5 whose salary is between $30,000 and $40,000.

SELECT * **FROM** EMPLOYEE **WHERE** (Salary **BETWEEN** 30000 **AND** 40000) **AND** Dno = 5;

The condition (Salary **BETWEEN** 30000 **AND** 40000) is equivalent to the condition((Salary >= 30000) **AND** (Salary <= 40000)).

Ordering of Query Results

SQL allows the user to order the tuples in the result of a query by the values of one or more of the attributes that appear in the query result, by using the **ORDER BY** clause.

Example:Retrieve a list of employees and the projects they are working on, ordered by department and, within each department, ordered alphabetically bylast name, then first name.

```
SELECT D.Dname, E.Lname, E.Fname, P.Pname
FROM DEPARTMENT D, EMPLOYEE E, WORKS_ON W, PROJECT P
WHERE D.Dnumber= E.Dno AND E.Ssn= W.Essn AND W.Pno= P.Pnumber
ORDER BY D.Dname, E.Lname, E.Fname;
```

The default order is in ascending order of values.We can specify the keyword **DESC** if we want to see the result in a descending order of values. The keyword ASC can be used to specify ascending order explicitly. For example, if we want descending alphabetical order on Dname and ascending order on Lname, Fname, the ORDER BY clause can be written as

```
ORDER BY D.Dname DESC, E.Lname ASC, E.Fname ASC
```

INSERT, DELETE, and UPDATE Statements in SQL

The INSERT Command

INSERT is used to add a single tuple to a relation. We must specify the relation name and a list of values for the tuple. The values should be listed *in the same order* in which the corresponding attributes were specified in the CREATE TABLE command.

Example:

```
INSERT INTO EMPLOYEE VALUES ( 'Richard', 'K', 'Marini', '653298653', '1962-12-30', '98 Oak Forest, Katy, TX', 'M', 37000, '653298653', 4 );

INSERT INTO EMPLOYEE (Fname, Lname, Dno, Ssn)
VALUES ('Richard', 'Marini', 4, '653298653');
```

A second form of the INSERT statement allows the user to specify explicit attribute names that correspond to the values provided in the INSERT command. The values must include all attributes with NOT NULL specification and no default value. Attributes with NULL allowed or DEFAULT values are the ones that can be left out.

A variation of the INSERT command inserts multiple tuples into a relation in conjunction with creating the relation and loading it with the *result of a query*. For example, to create a temporary table that has the employee last name, project name, and hours per week for each employee working on a project, we can write the statements in U3A and U3B:

U3A: CREATE TABLE WORKS_ON_INFO(

Emp_name VARCHAR(15),

Proj_name VARCHAR(15),

Hours_per_week DECIMAL(3,1));

U3B: INSERT INTO WORKS_ON_INFO

(Emp_name, Proj_name,Hours_per_week)

SELECT E.Lname, P.Pname, W.Hours

FROM PROJECT P, WORKS_ON W, EMPLOYEE E

WHERE P.Pnumber=W.Pno **AND** W.Essn=E.Ssn;

A table WORKS_ON_INFO is created by U3A and is loaded with the joined information retrieved from the database by the query in U3B. We can now query WORKS_ON_INFO as we would any other relation;

The DELETE Command

The DELETE command removes tuples from a relation. It includes a WHERE clause, similar to that used in an SQL query, to select the tuples to be deleted. Tuples are explicitly deleted from only one table at a time. The deletion may propagate to tuples in other relations if *referential triggered actions* are specified in the referential integrity constraints of the DDL.

Example:

DELETE FROM EMPLOYEE **WHERE** Lname='Brown';

Depending on the number of tuples selected by the condition in the WHERE clause, zero, one, or several tuples can be deleted by a single DELETE command. A missing WHERE clause specifies that all tuples in the relation are to be deleted; however, the table remains in the database as an empty table.

The UPDATE Command

The UPDATE command is used to modify attribute values of one or more selected Tuples.An additional SET clause in the UPDATE command specifies the attributes to be modified and their new values. For example, to change the location and controlling department number of project number 10 to 'Bellaire' and 5, respectively, we use

UPDATE PROJECT **SET** Plocation = 'Bellaire', Dnum = 5 **WHERE** Pnumber=10;

As in the DELETE command, a WHERE clause in the UPDATE command selects the tuples to be modified from a single relation. However, updating a primary key value may propagate to the foreign key values of tuples in other relations if such a referential triggered action is specified in the referential integrity constraints of the DDL.

Several tuples can be modified with a single UPDATE command. An example is to give all employees in the 'Research' department a 10 percent raise in salary, as shown by the following query

UPDATE EMPLOYEE
SET Salary = Salary * 1.1
WHERE Dno = 5;

Each UPDATE command explicitly refers to a single relation only. To modify multiple relations, we must issue several UPDATE commands.

Additional Features of SQL

- SQL has various techniques for specifying complex retrieval queries, including nested queries, aggregate functions, grouping, joined tables, outer joins, and recursive queries; SQL views, triggers, and assertions; and commands for schema modification.
- SQL has various techniques for writing programs in various programming languages that include SQL statements to access one or more databases.
- SQL has transaction control commands. These are used to specify units of database processing for concurrency control and recovery purposes.
- SQL has language constructs for specifying the *granting and revoking of privileges* to users.
- SQL has language constructs for creating triggers. These are generally referred to as **active database** techniques, since they specify actions that are automatically triggered by events such as database updates.
- SQL has incorporated many features from object-oriented models to have more powerful capabilities, leading to enhanced relational systems known as **object-relational**.
- SQL and relational databases can interact with new technologies such as XML

21. Briefly discuss how the different updata operations on a relation deal with constraint violations?

22. Consider the following schema for a COMPANY database:

EMPLOYEE (Fname, Lname, Ssn, Address, Super-ssn, Salary, Dno)

DEPARTMENT (Dname, Dnumber, Mgr-ssn, Mgr-start-date)

DEPT-LOCATIONS (Dnumber, Dlocation)

PROJECT (Pname, Pnumber, Plocation, Dnum)

WORKS-ON (Ess!!, Pno, Hours)

DEPENDENT (Essn, Dependent-name, Sex, Bdate, Relationship)

Write the queries in relational algebra.

i) Retrieve the name and address of all emp loyees who work for' Sales' department.

ii) Find the names of employees who work on all the projects controlled by the department number 3.

iii) List the names of all employees with two or more dependents.

iv) Retrieve the names of employees who have no dependents.